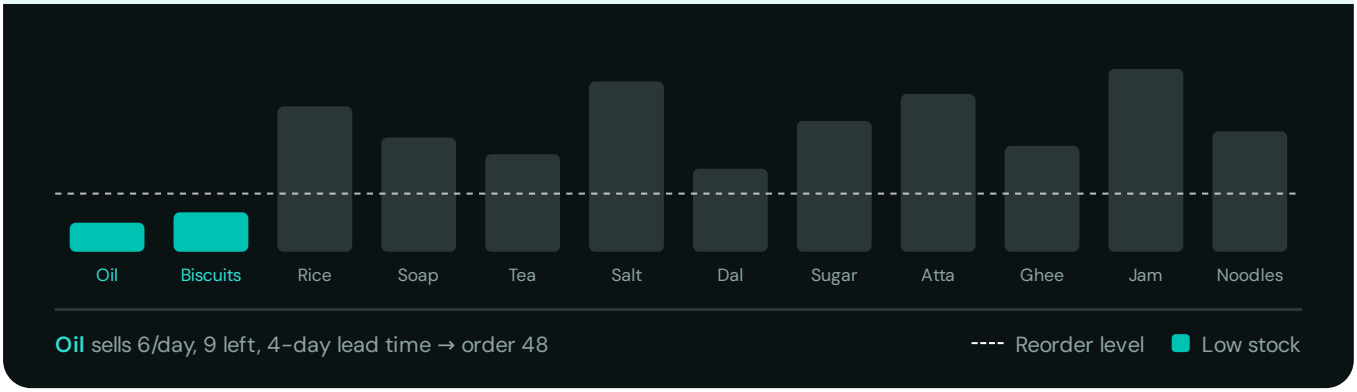
StockPilot AI

Smart Inventory Management

4-hour AI-assisted development challenge



4 hours	Solo	AI tools	OCR + forecast
to build & submit	any tech stack	allowed	must include

00 What to build

Official problem statement

Develop an inventory management system that helps small businesses manage their product stock and avoid overstocking or running out of products.

The system should

1. Allow businesses to add and manage products with their names, prices, and available quantities.
2. Allow users to upload purchase invoices and extract product details using OCR.
3. Track product sales and automatically update available stock.
4. Identify products with low stock and predict when they may run out.
5. Recommend which products need restocking and suggest quantities using sales history.

Guidance for building

1. **Build the frontend:** a dashboard to view products, stock levels and alerts.
2. **Develop the backend:** APIs to add products, record sales and update inventory.
3. **Integrate OCR:** extract product details from uploaded invoices.

Where each requirement is detailed 1 Products → §02 | 2 OCR → §03 | 3 Sales → §02, §05 | 4 Prediction → §04 | 5 Restock → §04

01

The problem

Ramesh runs a general store in Vadodara with about 300 products. He buys stock from four suppliers and records everything in a notebook. Fast sellers like cooking oil and biscuits run out before he notices, while slow items sit on the shelf for months. Supplier invoices pile up and are typed in by hand, so his stock count is almost always wrong.

Your task

Build a working website where a business manages products, imports purchase invoices with OCR, records sales and gets restock recommendations. Stock must **never go negative or change without a recorded reason**. Every recommendation must be **calculated from real sales history and explain why**.

People and businesses to demonstrate

Person	Role	Business	What they need
Ramesh	Owner	Ramesh General Store	Add products, upload invoices, see alerts and restock suggestions, set reorder levels
Priya	Staff	Ramesh General Store	Record sales and view stock; cannot change prices, delete products or approve invoices
Anita	Owner	Anita Medical	Manage a separate inventory; must never see Ramesh's data

These are example accounts for testing. Your website must let users create and edit products, invoices and sales — do not hardcode stock levels, alerts, predictions or recommendations.

02

Stock rules

Stock is a ledger, not a number someone types over: every change is a recorded movement with a reason.

Rule	Required behaviour
Products	Each product has a name, SKU (unique per business), unit (pcs, kg, litre, box), selling price, cost price, current quantity and reorder level.
Prices	Store money in paise (integer) or a decimal type, never floating point. Display in ₹ with two decimals.
Stock movements	Quantity changes only through a movement: purchase, sale, return, or manual adjustment. Each records who, when, quantity and reason.
No negative stock	A sale larger than available stock is rejected. Fractional quantities are allowed only for units like kg or litre.
Concurrent sales	Two staff selling the last 3 units at the same moment must not both succeed; the server re-checks stock in one transaction.
Repeated requests	Submitting the same sale or invoice twice must not change stock twice.
Deleting products	A product with stock history is archived, not deleted; its past sales and movements stay intact.
Manual adjustments	Owner only, and a reason is required (damaged, expired, counting error).

History	Keep an audit log of every product edit, invoice import, sale and adjustment.

03

OCR suggests; the owner confirms

OCR invoice import

- Nothing extracted from an invoice may change stock until a human reviews it.

1	**Upload**	Upload a purchase invoice as JPG, PNG or PDF (max 5 MB). Reject other file types with a clear message.
2	**Extract**	Extract supplier name, invoice number, invoice date and line items (product name, quantity, unit price, line total). Any OCR engine or vision model is allowed.
3	**Match**	Match each line to an existing product by name or SKU, or mark it as a new product. Show the match so the owner can change it.
4	**Review**	Review in an editable table. Highlight fields the system is unsure of, and flag when line totals do not add up to the invoice total.
5	**Confirm**	Confirm to create purchase movements in one transaction: all lines are added to stock, or none are.

What “working” means

- ✓ Upload a printed invoice photo; the review screen shows extracted lines that the owner can correct before saving.
- ✓ Confirm it; the matching products' stock increases by exactly the confirmed quantities, and the invoice appears in history.
- ✓ Upload the same invoice again; the system warns that this supplier and invoice number were already imported.

Use your own sample or mock invoices for testing. Do not use real supplier or customer data.

04

Predict, then recommend

Every prediction must come from the product’s own sales history and show its working. A simple, explainable method beats a complex one nobody can check.

Concept	Required behaviour
Sales velocity	Average units sold per day over a recent window (for example, the last 14 or 30 days). Weighting recent days more is allowed.
Days of cover	Current stock ÷ sales velocity. Show the predicted stock-out date.
Supplier lead time	Each product (or supplier) has a lead time in days, editable by the owner.
Low-stock alert	Raised when stock ≤ reorder level, or when days of cover ≤ lead time. Alerts clear automatically after restock.
Recommended quantity	Enough to cover lead time plus a target number of days, minus current stock, rounded up to the pack size. Never negative.

Explanation	Each recommendation shows its inputs (example below).
Too little data	Fewer than 7 days of sales → label “Not enough history” instead of guessing.
Bulk history	Allow a CSV import of past sales so judges can test forecasts without typing 30 days of entries.

Example explanation

Sells 6/day, 9 left, 4-day lead time → order 48 (2 cartons of 24)

05 Your website must

1. Provide owner and staff login with role-based dashboards. Enforce roles and business isolation on the server, not only by hiding UI buttons.
2. Let owners add, edit, search and archive products, filtered by name, category and stock status (in stock, low, out).
3. Import purchase invoices through OCR with a review-and-confirm step before any stock changes.
4. Let staff record sales (single item or a multi-item bill) and update stock automatically and atomically.
5. Show a dashboard with total products, low-stock count, out-of-stock count and today's sales at a glance.
6. Flag low-stock products and show each one's predicted stock-out date, calculated from sales history.
7. Recommend restock quantities with a visible explanation, and let the owner export the list (CSV or printable) to send to a supplier.
8. Show useful empty, loading, confirmation and error states. Reject invalid quantities, prices, files and unauthorised requests in plain language.
9. Keep the website usable on a phone: staff should record a sale in under 15 seconds, and the owner should see what to reorder on the first screen.

06 What happens when things go wrong

Situation		Expected result
Two staff sell the last units of a product at the same time	→	Only one sale succeeds; the other sees that stock is insufficient.
Anita changes a product ID in the URL to view Ramesh's product	→	The server denies access; no data from another business is exposed.
The same invoice is uploaded twice	→	Warn before import; confirming it again must not double the stock.

07

What to submit

Submit all items below through the Google Form: **forms.gle/H8dHiUdMusJA5n4E7**. Submission is a required part of the challenge.

1 **Source code (.zip)** — Due at 5:00 pm

Website plus a README with setup and run instructions, the forecasting method used, and the OCR tool used. Exclude secrets, .env files and large dependency folders (`node_modules` , virtualenvs).

2 **Working prototype** — Due at 5:00 pm

A deployed link judges can open, with demo credentials for owner and staff roles, plus 2–3 sample invoice images to test OCR.

3 **Slides (PDF)** — Due at 5:00 pm

PDF, max 8 slides that simply explain what you built to stakeholders. Suggested topics:

Problem · Solution · Screens · OCR flow · Forecast logic · Edge cases · Hardest bug · Improvements

4 **Product demo video** — Due at 5:00 pm

2–3 minutes. Show adding a product, an invoice upload and review, recording sales, a low-stock alert with its stock-out date, a restock recommendation, and at least one failure case.

5 **LinkedIn post** — Due at 5:00 pm

Share your experience: the StockPilot AI problem you tackled, your approach and the key moments along the way. It must include your product demo video or its link. Make it as creative as you can. Tag MedaVida, **Christina Camacho** (CEO), **Mallikarjuna Mudiam** (CTO), **Mahesh Das** (PM) and the team members.

Timing and rules

Build and post, by 5:00 pm
Code, live link, slides, video and your LinkedIn post link are all due by 5:00 pm. Late submissions are disqualified. Finishing early earns brownie points.

Present, right after
10 minutes each, with live questions on your code. Shortlisted participants get an email before end of day and move on to a technical interview with our CTO.

Engagement, until end of day
Likes, comments and reposts on your LinkedIn post are counted until end of day.

Solo work
Any language, framework or AI coding tool.

Original build
No prebuilt project passed off as new work.

No real data
No real personal, patient, business or customer data, and no exposed API keys or passwords.

Code freeze
Submitted code and website must not change after the deadline.

Bring your own
Laptop, internet and AI tool.

Here's what makes it exciting

The three posts with the most likes, comments and reposts by end of day will be featured on MedaVida's official LinkedIn page, and the top 3 participants with the most engagement win an exclusive 1:1 interview with our CTO.

Make the most of your 4 hours

A suggested plan. Adjust it to suit your build, but protect the last hour for shipping.



Design and development 1:00 – 3:30 pm	2 hr 30 min	Spend the first 15 minutes planning: pick your must-have features, sketch the main screens and decide what you will leave out. Get one complete flow working end to end before adding anything extra.
Deployment 3:30 – 4:00 pm	30 min	Put a bare version online in your first hour, so hosting surprises show up early. Then this slot is just a redeploy and a final check on your phone.
Video and slides 4:00 – 4:30 pm	30 min	Write a short script, record one clean run of your demo and keep it to 2–3 minutes. Build the slides from what you just showed.
LinkedIn post 4:30 – 4:40 pm	10 min	Draft it while your video uploads. Share what you built, one hard moment and what you learned.
Submission 4:40 – 4:45 pm	5 min	Fill in the form and check every link opens in a private browser window.
Buffer 4:45 – 5:00 pm	15 min	Keep this free for slow uploads or a last fix. Anything after 5:00 pm is disqualified.

Good luck — build something you'd trust to run your own shop.

medavida